Freely distributable tools for finding web vulnerabilities

1. Introduction

In this project I will be attempting to measure most some of the popular freely distributable open source tools for finding vulnerabilities in web applications. Here I will comparing tools based on how they perform in tests on 3 different appplications in 3 different activities. Emphasis will be put on number of true positives returned when executed. Activities include detection of sqli , unmaintained components and finding files accessible to external parties.

2. theoretical part

Tools selected are both specialized tools and blackbox tools. Specialized tools are programs made for one specific task. Black-box tools on the other hand are made to be capable of detecting all kinds of vulnerabilities. Since their focus is spread over a large area and each have very different implementations I will be running tests to compare their peformance and effectivness. While performing tests for hidden file detetion a specialized tool will be tested as well to show the differences between blackbox and specialized tool.

3. Scope of project

determine goals, constraints , strategies, tasks, and deliverables

4. project detail

5. Volunerabilities

By definition Security vulnerabilities are defined as weaknesses in technology, software, or operational practices that can be exploited by threats, leading to potential risks such as unauthorized access or information leakage [1]. These vulnerabilities can tak on various forms and in this paper I will be focusing on open source tools designed for finding flaws in networks and web applications.

6. Classification

Security vulnerability can be categorized into four categories those being software vulnerabilities, hardware vulnerabilities, configuration and operational vulnerabilities, and physical and personnel vulnerabilities [1]. Each of these catogiries has many sub categories and here I will be focusing only on network and web based software vulnerabilities. Classification of Common Vulnerabilities and Exposures(CVE) is provided by NVD which is using Common Weakness Enumeration(CWE) classification mechanism that differentiates CVEs by the type of vulnerability they represent. This classification distributes CVEs into a hierarchical structure CWE on higher level provide overview of vulnerability type and can have many subtypes. CWE are classified based on their nature and effect.

Main types of CVE this paper will be focusing on will be

- CWE-89: Improper Neutralization of Special Elements used in an SQL Command (‘SQL Injection’)
- CWE-552: Files or Directories Accessible to External Parties
- CWE-1104: Use of Unmaintained Third Party Components

All of which you can read more about on official nvd website.

7. Characteristics

7.1. CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Sqli injection is a common mistake make which is made by using unsanitized user input concatenated into SQL, unsafe ORM usage, dynamic query building without param binding. Impact of this mistake includes but isn't limited to unauthorized data access, modification, authentication bypass. Detection: injection payloads, error message analysis, blind/time-based tests.

7.2. CWE-552: Files or Directories Accessible to External Parties

This CWE is a relatively common mistake for beginners and missconfiguration of a server that allows unathorized user to access files not meant for them. This can affect any type of web server, ftp or similar server and is nearly always caused by missconfigured authorization. Most common way of abusing this missconfiguration is by using chroot() function. Detection: forced directory enumeration, public file checks, crawling for common backup/hidden file names.

7.3. CWE-1104: Use of Unmaintained Third Party Components

Common missmanagement mistake which happens when server admin does not properly update components and third party libraries of their application. Reliance on these components may result in third pary exploting vulnerabilities in these components which afterwards may result in any kind of damage from data leakage to whole servers or networks being compromised. Detection: static analysis of dependency manifests, CVE database matching.

8. Tools

Dirsearch – Specialized hidden file & directory enumeration w3af – Full-featured web vulnerability scanner (extensible plug-in architecture) Nikto – High-performance server scanner with large signature database Nuclei – One of the most popular open source projects of this sort Wapiti – Black-box scanner focusing on injection and file disclosure flaws Zap – GUI-based scanner developed by owasp Grabber – Lightweight scanner for small web applications

9. Experiments

Experiments will be done

10. scope of experiments

11. Testing Environment

First test will be conducted on 3 targets.

First target is a very faulty website that I coded myself. Whole project can be found on github **https://github.com/tomasswaier/infinityFreeWebsite/tree/c2569b1d7e5e77da0d68f011babb96452e250ba0** and link already contains exact sha I will be using as well. This website is faulty in multiple ways but 2 most important ones are sql injections in basically all php files and hidden files

11.1. SQLi

Thjis project uses unsanitized user input directly in string concatendate sql injections making it very visibly vulnerable.

11.2. Hidden files

For the purpose of testing hidden file discovery I've added to my vulnerable website couple of suspicious hidden files. These files are

“.env”,“admin.db”,“admin.html”,“db.log”,“kernel.conf”,“output.json”,“.logs/user.log”,“admin/.passwords/passwords.db”,“admin/adds.dev”,“admin/scala.run”,“config/config”,“config/config.log”,“config/config.php”,“config/initiate_connection.php” and some which should be harder ot detect :“dbx309183vbg$.out”,“presentation.pptx”,“quiyana.log”,“userLog321393029gx.txt”,“x321.log”. For the purpose of this project all .log files have been filled with data from real project.

Second target is custom made laravel application. This application should be an example of well made and secure application with no vulnerabilities. It will serve as a test for applications to see if they will be still able to find vulnerability even in a well secured environment.

Third targett is an open source project called Damn Vulnerable Web Application(DVWA) which is used specifically for these same purposes. It’s an app designed to be vulnerable in every way possible so that security experts may test their skills against it.

Optimal configuration used :

nitko: dirsearch -Tuning 123457b9 -mutate 6 -mutate-options wordfile= /common.txt -nointeractive

wapiti: dirsearch -u **http://localhost/** -m file –scope folder –scan-force aggressive –timeout 10 –max-scan-time 0 -f html -o /arch/second_year/pib/outputs/wapitiWebsiteOptDirS

dirsearch -u **http://localhost:8000/**
–extensions php,html,htm,txt,log,conf,env,ini,json,xml,bak,zip,old,sql
–exclude-status 404
–recursive
–deep-recursive
–max-recursion-depth 5
–force-recursive
–threads 50
–full-url
–random-agent
–no-color
–format json
–output results.json

zap These file types were added to the settings :php, html, js, bak, txt, zip, tar, conf, env,log,out,json Options force browse files without extension and force browse files were turned on Program was able to find the most relevant files config/initiate_connection.php /config.php /local.php /config test/get_data.php /load_tests.php

Nuclei Optimal configuration for nuclei includes adding templates for exposures,configs,files and logs.

Results:

nikto config, admin , test, .git and .env

wapiti 0 potentially dangerous files

zap Program was able to find the most relevant files config/initiate_connection.php /config.php /local.php /config test/get_data.php /load_tests.php nuceli This configuration resulted in nuclei finding .git , /config/ and .env.

12. Tools For Finding Web Vulnerabilities

applications to describe Dirsearch,w3af,nikto,nuclei,Wapiti,zap,Grabber

13. w3af

w3af includes csrf testing

14. Result Evaluation

Bibliography

[1] "Security Vulnerability." [Online]. Available: **https://www.sciencedirect.com/topics/computer-science/security-vulnerability**